SBA301 - Workshop 1 - RESTful CRUD API

Term SUMMER 2026

Deadline: follow the info shown on Classroom

Total points: 100

1 Instructions

- A database script **26__2__su26__ws1.sql** is provided. Run the script on your **SQL Server 2019+** instance to create the database `sba301_ws1_mojtsachs` and seed the sample data **before** running your Spring Boot application.
- Build a **fresh Spring Boot project** from scratch.
- Use **database-first** approach: map your Java entities to the provided tables using `@Table` and `@Column` annotations.
- Test all endpoints using **Postman** before submitting.
- Submit your source code on Classroom.

2 Requirements

2.1 Context

- You are building a RESTful API for a bookstore management system.
- The database `sba301_ws1_mojtsachs` has been pre-created via the provided SQL script. It contains two related tables: `author` and `book` (one author has many books).

Method	URL	Description
GET	`/api/authors`	Get all authors
GET	`/api/authors/{id}`	Get author by ID
POST	`/api/authors`	Create a new author
DELETE	`/api/authors/{id}`	Delete an author
GET	`/api/books`	Get all books
GET	`/api/books/{id}`	Get book by ID
GET	`/api/authors/{id}/books`	Get all books by an author
GET	`/api/authors/{id}/stats`	Get statistics for an author

2.2 Database Schema

Column	Type	Note
`author`		
`author_id`	`INT IDENTITY`	Primary key
`name`	`NVARCHAR(100)`	Not null
`nationality`	`NVARCHAR(50)`	
`birth_year`	`INT`	
`book`		
`book_id`	`INT IDENTITY`	Primary key
`title`	`NVARCHAR(200)`	Not null
`price`	`FLOAT`	Not null
`publish_year`	`INT`	
`author_id`	`INT`	Foreign key → `author`

3 Tasks

3.1 Project Setup

- Create a new Spring Boot project (via IntelliJ IDEA or `start.spring.io`) with **Java 21+** and the following dependencies: **Spring Web**, **Spring Data JPA**, **MS SQL Server Driver**. Lombok is optional but recommended.
- Configure `application.properties` for the database connection. Organise your code into the following package structure:

```
vn.edu.fpt.sba/
  +-- entity/
  +-- repository/
  +-- service/
  +-- controller/
```

3.2 Entity Layer

1. Implement the `Author` entity class. Map it to the `author` table using `@Entity`, `@Table(name = "author")`, and `@Column(name = "...")` annotations so that every field matches the column names exactly. [8 points]
2. Implement the `Book` entity class. Map it to the `book` table with proper `@Table` and `@Column` annotations. Declare the many-to-one relationship to `Author` using `@ManyToOne` and `@JoinColumn(name = "author_id")`. [12 points]

3.3 Repository & Service Layer

1. Create `AuthorRepository` and `BookRepository` for the two entities. [5 points]
2. Create `IAuthorService` interface and `AuthorServiceImpl` to handle author-related business logic (CRUD operations). [10 points]
3. Create `IBookService` interface and `BookServiceImpl` to handle book-related business logic. [10 points]

3.4 Author Controller

1. Implement **GET** `/api/authors`. Return the list of all authors with HTTP **200 OK**. [5 points]

   Expected response (200 OK):

```
[
  { "authorId": 1, "name": "Nguyen Nhat Anh",
    "nationality": "Vietnamese", "birthYear": 1955 },
  { "authorId": 2, "name": "J.K. Rowling",
    "nationality": "British", "birthYear": 1965 }
]
```

2. Implement **GET** `/api/authors/{id}`. Return the author with HTTP **200 OK**. If the author does not exist, return **404 Not Found**. [5 points]

   Expected response (404 Not Found):

```
{
  "error": "Author not found with id: 99",
  "timestamp": "2026-06-09T10:30:00"
}
```

3. Implement **POST** `/api/authors`. Accept a JSON body, persist the new author, and return **201 Created** with the saved object (the `authorId` is auto-generated by the database). [10 points]

   Request body:

```
{ "name": "Haruki Murakami",
  "nationality": "Japanese", "birthYear": 1949 }
```

Response (201 Created) — `authorId` value may vary:

```
{ "authorId": 5, "name": "Haruki Murakami",
  "nationality": "Japanese", "birthYear": 1949 }
```

4. Implement **DELETE** `/api/authors/{id}`. Delete the author and return **204 No Content** (no response body). [5 points]

5. Implement **GET** `/api/authors/{id}/books`. Return all books belonging to the given author. [10 points]

   - If the author does not exist → **404 Not Found**.
   - If the author exists but has no books → **200 OK** with an empty array `[]`.

   Expected response (200 OK) for `/api/authors/1/books`:

```
[
  { "bookId": 1, "title": "Mat Biec",
    "price": 95000.0, "publishYear": 1990,
    "author": { "authorId": 1, ... } },
  { "bookId": 2,
    "title": "Cho Toi Xin Mot Ve Di Tuoi Tho",
    "price": 85000.0, "publishYear": 2008,
    "author": { "authorId": 1, ... } }
]
```

6. Implement **GET** `/api/authors/{id}/stats`. Return a statistics object for the given author. If the author does not exist, return **404 Not Found**. [10 points]

   The response must include the following computed fields:

   - **bookCount**: total number of books by this author.
   - **totalValue**: sum of the `price` of all books.
   - **averagePrice**: *totalValuebookCount*, or `0.0` if the author has no books.

   Expected response (200 OK) for `/api/authors/1/stats`:

```
{
  "authorId": 1,
  "authorName": "Nguyen Nhat Anh",
  "bookCount": 2,
  "totalValue": 180000.0,
  "averagePrice": 90000.0
}
```

3.5 Book Controller

1. Implement **GET** `/api/books`. Return the list of all books with HTTP **200 OK**. Each book object must include the nested `author` object. [5 points]

   Expected response (200 OK):

```
[
  { "bookId": 1, "title": "Mat Biec",
    "price": 95000.0, "publishYear": 1990,
    "author": { "authorId": 1,
                "name": "Nguyen Nhat Anh",
                "nationality": "Vietnamese",
                "birthYear": 1955 } },
  ...
]
```

2. Implement **GET** `/api/books/{id}`. Return the book with HTTP **200 OK**. If the book does not exist, [5 points] return **404 Not Found**.

   Expected response (404 Not Found):

```
{
  "error": "Book not found with id: 99",
  "timestamp": "2026-06-09T10:30:00"
}
```

— THE END —